What Computation Costs

Software engineering as the work of an interpreter
standing between a person and a Turing machine

Edwin O Onyango
edwin.o.onyango.jr@gmail.com

Walter O Wagunde
wagudewalter2@gmail.com

September 2026

Abstract

In 1936 Turing described a computer by watching a person do arithmetic and setting every cost that person pays to zero. Software engineering then grew up between the machine and the people who want things from it, and this paper describes software engineering from the outside. Code is the layer through which people tell the machine what to do. A software engineer is an interpreter, standing between a person who wants something and a machine that does exactly what its sheet says. The interpreter's work has three tiers: finding out what is wanted, turning it into a *software intention*, and turning the intention into code. Code generators are absorbing the third tier. Software engineering keeps most of what it knows in the second tier. It keeps it as rules, and the conditions under which each rule holds were dropped as the rule was passed on, which is why every experienced engineer eventually learns that it always depends.

The claim of the paper is that those dropped conditions can be recovered. They were never anything but the machine's constraints, and the machine's constraints come in two kinds. A few things it cannot do at any price; those are walls, and all but one are theorems. Everything else it can do at a price, and the prices are exactly the eight costs Turing idealised away. The eight sort into three groups. Five are physical, and physics sets floors under them. One is the boundary between the machine and the people who write for it: somebody has to write the program. Two are the world: the input comes from people, and the job never ends. The field's cost models stop after the physical five, its theorems stop after the boundary one, and beyond that line there are only rules. In Turing's machine every move is completely determined by the state and the symbol under the head; nothing is ever chosen, because nothing costs anything, and a choice exists only where two options cost different amounts. So every engineering decision is a choice between alternatives that pay differently under one or more of the eight. That gives every decision a shape, and every rule a test: what is it answerable to? A rule can answer to a cost, to a theorem, to a measurement, or to nothing but the fact that others follow it. Put to four rules everyone knows, the test finds one resting on a formula that has been recomputed for forty years, one resting on a cost its own author named and nobody quotes, one resting on a theorem that cannot license it, and one whose number is the width of a 1928 punched card, kept because everyone keeps it. A decision itself has a cost with four parts: writing the change, checking it, the chance it is wrong times the cost of undoing it, and what is spent finding out before committing. Code generators remove the first part. The three that remain are all about being wrong, so what has to be verified is the decision, not the code. The paper ends with seven predictions and an invitation to find a principle that does not fit.

---

1 The Machine

In 1936 a "computer" was a person at a desk with paper, a pencil and a printed sheet of instructions, doing work that astronomers, insurers and the makers of artillery tables depended on. Turing was answering Hilbert's question of whether a mechanical procedure could settle any mathematical statement, and had to make "mechanical" precise. He did it by describing

what that person does [95, 45], and the description was exact enough that he later defined a digital computer against it [96]. There is a tape of squares, a head over one square, a finite state, and a sheet that says, for each state and symbol, what to write, which way to move, and what state to enter.

Is that all a person computing by rule ever does? Turing claimed so. The claim cannot be proved, only searched for exceptions, and the search has failed. Every other definition of computation offered before or since computes the same class; Church's calculus and Gödel's general recursive functions, which Kleene showed equivalent, were among the first [19, 54]. That is why it is the Church-Turing *thesis*. Of those definitions, Turing's is the one that describes a person: the lambda calculus has no tape and no head, hence no notion of where a value is or what reaching it costs, and that is the feature that made Turing's analysis persuasive where Church's was not [33, 86, 90]. The paper uses the thesis twice. Read, write, move, change state and halt are taken as the whole of what any machine does; those are the five operations in the model's modern statement, where halting is a state, and Turing's own machines printed for ever. And a claim held by a failed search for exceptions is the standard the paper's own lists are held to.

Every operation costs the clerk something, and the desk imposes five costs the model omits: paper must be bought, a distant square takes longer to reach than the next one, one person sits at the desk, paper smudges and memory fails, and anyone who walks past can read what is on it. Three more things cost him and are free in the model. Somebody wrote the sheet. Somebody decided what goes on the tape. And the work is never done once, since a table is corrected for as long as anyone uses it, whereas in the model the job is fixed when the machine starts. Turing set all of it to zero because what is computable cannot depend on how fast a pencil moves.

Philosophy of science has a name for this move. An *idealisation* is a real quantity assumed to be zero so that a theory becomes tractable: the frictionless plane, the point mass, the ideal gas. Putting the quantity back afterwards, to see what it changes, is a method in its own right [65]. Turing's machine is the ideal gas of computing, and if the costs of computing are to be put back into one model, it has to be this one. To idealise a cost away Turing first had to assume something in its place, one tape, one head, a sheet already written, a job that is fixed, so the model is an inventory of assumptions, each standing in for a cost the clerk pays, and an inventory can be audited one line at a time. Dally [22], Valiant [98] and the Fallacies of Distributed Computing [30] each restored the costs they needed and stopped. This paper reads the audit to the end, and says what a complete audit is for.

One more feature of the model carries §9. In an automatic machine, in Turing's words, "the motion of a machine is completely determined by the configuration": the state and the symbol under the head fix the next move, and there is no point at which anything is chosen. Turing did allow a machine that at some configurations "cannot go on until some arbitrary choice has been made by an external operator", and set it aside at once, "in this paper I deal only with automatic machines" [95]; and the programmer, who chose what to write on the sheet, is outside the model altogether. So the model removes choice twice, and every decision a real engineer makes entered from one of two places: the writing of the sheet, or a cost the model set to zero. The real clerks were already paying under both. Prony decided how the sheet was written when he split arithmetic into steps unskilled workers could follow, and the remedy for errors in printed tables, two computers working the same sheet, is redundancy against a tape that fails [39]. Engineers decide the same things at speeds eight orders of magnitude higher, plus the one thing the clerks never decided: the sheet. How anyone writes a sheet is the next question.

2 Code

Nobody writes Turing's sheet. Code is a stack of translations that ends at it: framework, library, language, compiler, instruction set, circuit. Every layer exists for one reason, so that a person can say more per symbol written and read more per symbol read. The machine can already do anything any layer expresses. So the layers change nothing about what can be said; they change only what it costs a person to say it and to understand it. That cost is one of the eight the model removed; idealisation 6, below, is the assumption that hides it.

At every layer the meaning of a text is fixed. A language standard fixes what a text means by fixing what the machine must observably do, in the C standard's words "the observable behavior of the abstract machine" [17]. This is a claim about meaning, not about execution: a program may draw random numbers or run threads in any order, and that is meaning fixed as "any of these", on purpose. The defect the profession fences off is meaning left unfixed by accident, undefined behaviour, a race nobody intended. A learned model is a program in this sense too, fixed weights with fixed meaning on the same hardware, and what varies in its output is sampled on purpose. A text with one meaning can be checked without being run, and every practice of reading, reviewing, typing and proving code rests on that.

So code is a way of telling the machine what to do, in a form a person can write, that does exactly what it says. The computer collapsed the price of executing a procedure once it was specified; what code cannot supply is the specifying, what to say. The people who want something from the machine do not speak in sheets, and between them and the machine a software engineer has to stand.

3 The Interpreter

A software engineer is an interpreter, in the courtroom sense rather than the compiler sense. An interpreter renders meaning live between two parties who share no language, chooses words under the constraints of the second language, and is answerable to both sides for the result. On one side here is someone who wants something and can say so only in the terms of their own world: an invoice must not be altered after it is sent, a customer should not wait. On the other side is a machine that will do exactly what its sheet says, at the prices the world charges for each line, and that understands nothing else. Everything software engineering knows is knowledge about how to stand between them.

Before any code is written the engineer produces a statement of what the code must do, and the paper calls it a *software intention*: a statement of what the machine is to be told, in terms the machine can be told, made before the telling. It is not the want, which lives on the world side. It is not the code, which is one of many texts that could carry it. It is where the decisions live: which quantity must be preserved and at which level, what may be assumed about the world, what the machine will remember and for how long, where the boundaries fall, what happens when a part fails. The idea has neighbours. Simon described any artifact as a meeting point between an inner environment, its own substance, and an outer environment, the surroundings it must serve, "an interface in today's terms" [88]; this paper is about the engineer who builds the meeting point, and what the inner environment charges. Brooks's essence is the conceptual construct that is left when the accidents of representation are stripped away, and he predicted that only the accidents would shrink [15]. Jackson and Zave put a specification at the interface between world and machine, with the requirements on the world side [105]; the reference model built on that, world, requirements, specification, program, machine, is the layering the tiers below inherit [41]. Naur's theory is what the builders hold and the text does not [71]. The new name claims that the software intention is where the engineer's expertise is, and that it has coordinates, which §9 gives.

3.1 Three Tiers

The work has three tiers, and each is a different translation. The first tier goes from the world to the requirements: finding out what is wanted, what the people who want the software actually want, which constraints are real, what will change. The second tier goes from the requirements to the specification, and on to the design decisions the reference model leaves inside the program. It turns what is wanted into a software intention, and it is a negotiation with the machine, which can do a few things at no price whatever and everything else at a price. The third tier goes from there to the program text. It is the tier software engineering has spent most of its history on, and the tier code generators are absorbing. The layering is the reference model's. This paper says what shapes the second tier, and what happens to the third when it becomes free. The paper leaves the first tier alone; §7 says why.

The lines between the tiers move: the line between the second and the third is the cost of producing code, and §12 is about its collapse. What does not move is that the middle tier is where the decisions are, and that every decision there is made against the machine's constraints. So the paper needs two things: a sorting of what the interpreter already knows, and a complete list of the constraints. The first points at the second.

4 Three Kinds of Knowledge

Knowledge of software comes in three kinds, told apart by what happens when a piece is handed to somebody else. *Theorems* regenerate: whoever holds the reason can rebuild every consequence anywhere, because the conditions travel inside the statement. *Lookups* do not regenerate at all: a port number, a flag, the name of a method somebody else wrote. *Rules* sit between: index the column the query filters on, keep services small, pin the dependencies, do not optimise prematurely. A rule generates as a theorem does, one sentence covering a thousand situations, but it generates correctly only under conditions, and the conditions are not in the sentence. Whoever found the rule found it as the cheapest answer under a particular machine, team, business and set of prices. The answer was passed on and the conditions stayed behind, usually because they were too obvious at the time to state. The three form a spectrum, from statements that carry all their conditions to statements that carry none, and medicine has the same spectrum, a mechanism as theorem, a dosage as lookup, a clinical guideline as rule.

The spectrum is not an accident of the field. Knowledge accumulates where reality answers, and its form is set by how fast and how exactly reality answers. A program holds still, so a question about programs gets an answer that does not move, sometimes an exact "no", and that is a theorem. A fact that holds still but carries no reason, a port number, regenerates nothing and needs no theory; that is a lookup. A price moves, so a question about prices gets an answer that is true for a while, and that is a rule whose conditions are the prices at the time. What people want cannot be measured, so questions about it get no formal answer at all. Sections 7 and 13 check this against the record.

The middle of the spectrum is where the interpreter spends the week, and applying a rule there is expertise, not recall: the expert supplies the missing conditions from experience, and cannot always say why. Knuth's remark on premature optimisation is the clearest case (§10.1): the half that became a proverb has travelled for fifty years, and the cost it was weighing, two sentences earlier in the same paragraph, has not travelled at all.

The three must cover what an engineer knows, or what follows is partial, so the apparent exceptions get a sentence each. Taste is a rule whose conditions have been compiled beyond the holder's ability to state them (§11). A war story is a rule with a sample of one, and the rare case where the conditions travelled. A benchmark number is a lookup with a date. A design pattern is a rule published with its conditions attached, the one place the field has used

that form for fifty years [4, 32]. A contract is a lookup other people have agreed to keep true.

4.1 Method

The paper rests on one observation. A rule is the cheapest answer under some conditions, and the interpreter's conditions are the machine's constraints, because only the machine's charges can be written down, and the team and the business enter its account as the costs of writing the sheet and of the world moving. So the conditions that fell off every rule were statements about what the machine cannot do and what it charges, and once the complete list of constraints is recovered, the dropped conditions have somewhere to be looked up. The list must come from underneath the practice, since a list drawn from the practice is only the practice again; without it the exercise collapses into "engineering involves trade-offs", which forbids nothing. Software runs on computers, so whatever is true of every computer is true of every program; and the smallest true description of every computer is the clerk minus the costs.

The method is de-idealisation, done in full and then used: put each cost back, price it where physics can, sort the eight, and check the sorting against two counts, one of cost models and one of theorems; then derive the decision space, put four rules to the test from their primary sources, and check the result against the field's own history of practice reversals; then price a decision and remove one term. Claims are marked *proved*, *measured* or *disputed* where it matters, and every reading of a source is recorded in a companion document, named at the end. The constraints come first, and the first of them are the few things the machine cannot do at all.

5 Walls

Walls are few, and all but one are theorems. No machine can decide from another machine's sheet whether it will run without getting stuck or ever print a given symbol [95], and Rice generalised this to every non-trivial question about what a program computes [79]. A machine with finite memory is formally a finite-state machine whose halting is decidable by enumerating its states, at a price exponential in their number, so on a real machine the wall is met as a price nobody can pay, and it stands. So a sheet cannot in general be checked by reading it, and typing and testing are the compromises: a type system decides a property conservatively, a test samples behaviour at points a person chose. Where messages may be delayed without bound, no deterministic protocol can guarantee to reach agreement if one participant may fail [31]. A service that must survive a partition cannot, during the partition, be both consistent and available [34]. So every distributed design must choose what it gives up. And a machine running near the physical limit on its rate of operation cannot be serial (§6); that is the one wall that is a physical limit rather than a theorem.

Two things follow from how short the list is. First, a wall says "not that" and never says which of the remaining options to take, and the remaining options differ only in what they cost. Even CAP hands the decision to the prices, which decide whether consistency or availability is given up. Second, walls are the first kind of knowledge, theorems with their conditions attached, so a practice resting on one can be traced to its fine print. Everything else the machine can do, at a price, and the prices the universe itself sets come first.

6 Four Bills

A Turing machine costs nothing to define. A physical computer has to be built, powered, cooled and maintained, and the gap between the two comes apart into four costs, in the order

the model lists its operations: writing, which physics charges for as erasing; moving; the tape itself, which fails; and the passage of steps.

Erasing. Forcing an unknown bit to a known value halves the arrangements of the world consistent with what is known, and the second law makes that entropy appear as heat: at least $kT \ln 2$ per erased bit, about 2.9×10^{-21} joules at room temperature [57]. The cost attaches to erasing, not to computing, since any computation can run reversibly if a record is kept [10]. The floor is reached only for infinitely slow erasure [77]. It is *measured* [12, 51, 46, 103] and *disputed* at its foundations [28, 73], and nothing here depends on the dispute. Real machines run a million times above it. This is the famous bill and it bears on nothing anybody does.

Moving. The random access machine cannot be built. It is the model on which every $O(n \log n)$ is counted, and in it address three costs the same as address three billion. Memory cells have a minimum size and signals have a maximum speed. So the farthest of x cells packed around a processor in d dimensions is about $x^{1/d}$ away, the latency to reach the x-th cell is $\Omega(x^{1/d})$, and a pipelined hierarchical memory achieves that bound [13]. This is usually presented as a design heuristic, and it is not one. A program has a cache because geometry says it must, and no money removes it. Three pieces of fine print. The bound is on latency to address x, not on average latency. The proof leans on a density bound whose scope is still being worked out [43]. And it bounds latency, not throughput, so whether the hierarchy is visible to the programmer remains a choice; that is where physics ends and the designer's decision begins.

Failing. More than eight percent of memory modules in Google's fleet saw a correctable error in a year [81], and flash is comparable [68]. Below a threshold error rate, majority voting over redundant components buys arbitrary reliability at finite overhead [101]. The industry pays that overhead as error correction, replication and scrubbing.

Stepping. A physical system passes through distinguishable states no faster than a rate set by its energy [64]. For a kilogram of matter in a litre, the time for a signal to cross the machine exceeds the time to flip one bit by about 10^{10}, and only at black-hole density does the ratio approach one [63]. A machine near that limit must be parallel; that is the physical wall of §5. Every machine ever built runs far below the limit. At network scale the floor is the speed of light. New York to London and back is near 37 milliseconds in vacuum and 55 in fibre; measured pings are about three times the vacuum floor, and full fetches about thirty-six times [14]. The gap is engineering; the floor is not.

Turing grants for free	**The physical bill**	**Status**
Overwriting a cell is free	$kT \ln 2$ per erased bit, plus a term that grows as erasure is done faster	Proved; measured; foundations disputed
Any cell is reachable in unit time	$\Omega(x^{1/d})$ latency; a pipelined hierarchy is optimal	Proved [13]; verified here from the abstracts
The tape never fails	Redundancy and correction overhead; $> 8\%$ of DIMMs per year	Measured [81]; survivable by [101]
Steps follow one another without limit	A maximum state-change rate per unit energy; the speed of light; a communication-to-operation ratio near 10^{10}	Proved and measured [64, 63]

6.1 Floors and Prices

A *floor* is a cost nothing lowers, because it follows from the speed of light or the size of atoms. A *price* is a cost money moves, and prices move by orders of magnitude within a career. A practice resting on a floor is right for as long as anyone builds computers. A practice resting on a price has an expiry date, and the value of knowing which is which is seeing the expiry coming rather than finding it in a postmortem. The range of prices under which a practice is the cheaper choice is its *regime*.

Physics sets floors and almost never selects a design. Dennard scaling, that shrinking a transistor's dimensions and voltage together keeps power density constant [24], held until leakage ended it between 2007 and 2012 [44]. The sentence that usually follows, "so the industry had to go multicore", does not follow. Physics forbade a single processor getting faster at constant power and said nothing about the response; accelerators, better software or accepting the performance on offer were all available. Multicore was a choice, and reporting a chosen response as a physical necessity is the error this paper exists to catch. The four bills are the floor of the space in which engineering happens; the rest of the space is what the model left out.

7 Eight Idealisations

Take the model's assumptions one by one and ask what a real machine pays that the model does not. Eight answers come back, and those are the *idealisations*; to pay under an idealisation is to pay the cost it hides. They sort into three groups. The first five are physical. The sixth sits at the boundary between the program and the people who write it. The last two come from the world outside it. The two censuses that follow the list show where formal knowledge stops.

1. The tape is unbounded and free. A real machine is finite and a byte has a price, which has fallen about six orders of magnitude since 1987 and is the input to Gray's formula in §10. Generates the storage tiers as a price ladder, what is kept and what is recomputed, and the rule to keep the working set in memory.

2. Movement is free. False by the latency bound, and measured at every distance: sequential memory about half a nanosecond, random memory twenty, local flash a hundred microseconds, disk ten milliseconds, object storage eighty [70], eight orders of magnitude, so that if the fastest were one second the slowest would be three years; and within the processor a last-level cache miss costs about 250 core cycles on Skylake and 420 on Zen 4 [82, 18], the product of memory latency improving at about 7% a year against processor speed at 25 to 50% [44], foreseen in 1994 under the subtitle "implications of the obvious" [102]. Generates the memory hierarchy, caching, indexing, locality as a first-class concern, columnar storage, data-oriented design, and the rules every engineer knows: index the filtered column, cache what is expensive and stable.

3. There is one head. Real machines have many, because a machine near its rate limit must be parallel and machines far below it went parallel by choice (§6.1), and the heads must be coordinated. Measured, indirect costs dominate: a context switch 3.8 microseconds directly and hundreds through cache pollution [61]; a system call 150 cycles directly and about 12,300 in full footprint [91]. Generates concurrency control, transactions, isolation levels, memory models.

4. One tape, in one place, that does not fail. Failure is the third bill. Distance: an in-process call about a nanosecond, a system call three hundred, a same-region network call 250 microseconds [70], or two to four with kernel bypass [52]; three to eight orders of magnitude depending on the infrastructure. Generates distribution, protocols, replication, consensus.

5. The machine is alone. Idealisation 3 is many heads the designer controls; this is heads

the designer does not: other tenants, other authors, attackers. An attacker is somebody who reads a variable the designer declared irrelevant: Spectre read discarded timing, Meltdown read state the architecture declared invisible. So what changes here over time is not a price. It is whether the premise that some variable is irrelevant is still true. Generates isolation, capabilities, sandboxing, tenancy.

6. The program is given. The sheet is written by people of limited memory and attention and revised by others for as long as the system matters, and nobody holds the whole of it. Simon's argument that anything complex assembled in interruptible steps ends up a nearly decomposable hierarchy is the founding statement [87]. Generates languages as tools for people, abstraction, modularity, interfaces, version control, review, testing, architecture; nearly every rule in a style guide is a price paid for this cost.

7. The input is given. What a system should do lives in people's heads, incompletely, and is discovered while building; this is Naur's programming as theory building [71]. Generates requirements, product work, every method that shortens the loop between building and finding out whether it was wanted.

8. The job is fixed and finishes. Maintained systems never halt; they are changed while running, the world moves under them, and their structure degrades unless effort is spent against it [59]. Generates maintenance, operations, observability, migration, the economics of technical debt.

The machine in this list is not any particular modern computer. A processor, a phone, a data centre and a fleet are each Turing's machine, matter arranged so that its state changes correspond to the five operations for every input, and each has a different price list under the same eight; a new substrate changes the prices, never the list, and the walls of §5 do not move for any of them. Computation that never passes through symbols, the analogue kind, is set aside here as Turing set aside the choice machine.

The bills set floors under the physical five without pricing each one: the latency bound under 2, stepping under 3 and, as the speed of light, under 4, failing under 4; erasing binds nothing. The first has a market price and no floor that binds. The fifth has measured overheads, and behind it stands a premise rather than a price.

7.1 Two Censuses

Two counts, with different instruments, find where formal knowledge runs out. The first count is of cost models. The physical five have them, four with published constants and the fifth with measured overheads for isolation; the other three have none. The second count is of theorems, each sorted under the idealisation whose cost its question presupposes, so that a result about checking a sheet somebody wrote goes under 6 and a result about a tape in more than one place goes under 4. The durable results of computing pile up under 6 (undecidability, Rice, Myhill and Nerode, Church and Rosser, Codd, Hindley and Milner, Cook and Levin) and under 4 (FLP, CAP, Paxos, Raft, replicated data types). Under 5 the unconditional results are about what a foreign head can learn from what it reads, not about keeping it off the tape, and they are few. Under 7 the results are about learning functions from samples, not about eliciting intent from people. Under 8 there are none, since Lehman's laws are regularities, Conway's is an observation, and Brooks's is an argument.

The two world idealisations were enumerated, as interaction with an outside world, a program that changes and a machine that never stops, by van Leeuwen and Wiedermann [99]. This paper claims the full list, the sorting, and that the sorting predicts where formal knowledge runs out. Completeness is a failed search, refutable as Turing's thesis is: exhibit a ninth cost the model discards that is not a combination of these.

The list fixes the line between the interpreter's first two tiers. Idealisations 7 and 8 are the world, what people want and what they will want next; everything from 6 down is the machine and the people who write for it. The first tier works beyond that line, no cost model reaches it, and that is why the paper leaves it out. The idealisations say what a real machine pays. They do not yet say where, and where is the next question.

8 Rows

A *row* is the level of a system at which an operation is performed and paid for, from the machine to the institution. The five operations recur at every row, at prices separated by many orders of magnitude, and the rest of the paper refers to rows by name.

Row	Read	Write	Move	State	Halt
Machine	load	store	address	register	halt instruction
Language	evaluate	assign	control flow	scope	return
Program	call a getter	call a setter	follow a reference	object state	terminate
Process	receive	send	schedule	process state	exit
System	query	commit	route	session	shut down
Organisation	ask someone	tell someone	hand off	ownership	disband the team
Institution	read the spec	publish a version	migrate	the standard	deprecate

Read the *read* column down with prices: a register in a fraction of a nanosecond, main memory in tens of nanoseconds, flash in a hundred microseconds, another continent in tens of milliseconds. Then past the machine, where a read means asking a colleague: minutes at the next desk, days in another team, unavailable if they have left. That last step makes Conway's law a cost law: the organisation's communication graph is the price list for reads at that row [21].

The *halt* column matters more. Stopping something is one instruction at the machine row, a restart at the process row, a deployment with a rollback plan at the system row, a reorganisation at the organisation row, a deprecation announced years ahead at the institution row. *The cost of stopping rises with the row at which the decision was bound.* That one observation is the general form of much architectural advice: do not bake configuration into code, a business rule into a migration, a policy into a protocol, a team structure into a public API. Each of these says that a decision was bound at a row whose halt cost exceeds what it was worth, and each becomes checkable once the row is named. Many of the field's loudest arguments are two rows mistaken for one.

8.1 Where a Quantity Is Preserved

Because every operation exists at several rows, anything that must be preserved could be preserved at any of them, and choosing the row is a decision. The end-to-end argument addressed exactly this, and Saltzer, Reed and Clark stated the under-determination: the argument "does not tell us where to put the early checks", it is "a guideline", and they supplied the cost model that decides, with a crossover in file size beyond which end-to-end retry is wrong [80]. That is a rule published with its conditions.

Preserved quantity	Rows that could carry it
Uniqueness of a key	the database constraint, the data-access layer, the service, the client
Retry on failure	the transport, the client library, the application handler, the human
Type correctness	the compiler, the runtime, the schema registry, the test suite
Authorisation	the gateway, each service, the database, the query
Idempotency	the caller, the broker, the handler, the store
Ordering	the log, the queue, the consumer, the reconciliation job

An architectural decision is an assignment of a preserved quantity to a row, and its cost is the halt cost of that row. That is one kind of decision. Whether every decision has this shape is the next question.

9 Decisions

A decision exists only where two alternatives differ in cost. That is why nothing is ever chosen in Turing's machine. With every cost set to zero it does not matter where a write goes or how far away a cell is; nothing races, nothing is lost, nobody else is present, nobody has to be asked, nothing has to be maintained. So the paper need not list decisions, which would be hopeless; it inherits the enumeration from the idealisations.

> *Every decision in software engineering is a choice between alternatives that differ in what they pay under one or more of the eight idealisations, and a decision is complicated in proportion to how many it pays under at once.*

9.1 A Grammar of Decisions

Three coordinates suffice, and all three are already in the paper. The *operation* is a read, a write, a move, a state change or a halt, since those are the only things a machine does. The *row* sets the halt cost, which is what a decision costs when it is wrong. The *idealisation set* is the set of idealisations under which the alternatives differ; it is what the decision is answerable to, and it is the coordinate nobody records. A decision is a point in operation × row × idealisation set, and its *shape* is which point it occupies.

Decision as people say it	Operation	Row	Paid under
Index the column the query filters on	read	system	1, 2
Keep the working set in memory	read	system	1, 2
Make it a microservice	move	system, organisation	2, 3, 4, 6, 8
Add a boolean flag	state	program	6, 8
Pin the dependency version	write (by another head)	program, institution	5, 6, 8
Write a test for it	halt (a gate)	program	4, 6, 8
Ask the founder before building	read	organisation	7
Put it behind a feature flag	halt	system	8
Retry on failure	move	any of four rows	4
Rename this for clarity	write	language	6

The decisions everyone calls hard pay under many idealisations at once, and microservices, the worst offender, is five debates wearing one name, which is why it never ends. The decisions everyone calls trivial are often those whose idealisation set was misidentified, which is where the deceptively simple failures come from. The rest of the felt complexity is a row nobody

named or an idealisation set that moved with the prices, and naming the coordinates recovers both.

9.2 A Dependency Failure

A library ships a new version and the build breaks, or passes and production breaks. It looks like a lookup, resolved by reading a changelog, and it is not. A dependency is a region of the program's tape that a head its author does not control may write to, and the program's correctness rests on those symbols staying what they were. That is idealisation 5. It is composed with 6, since the sheet was written by people who did not know about this program, and with 8, since the world moves while the machine runs. "Keep dependencies up to date" addresses 6 alone, and the failure comes from the other two. Once that is visible, the resolution is three questions. Which symbols in the foreign region does the program actually read? That is the *cut*; semantic versioning is an attempt to write it down as a promise, and semver failures are the promise being narrower than the cut. At which row was the version bound? In a lockfile the halt cost is a one-line revert; in every service's base image, a coordinated redeploy; in a platform the team does not control, a deprecation cycle it does not own. Who else has a head over the region? Two dependencies reading incompatible symbols from a third have no fix at the program row; that is why diamond dependencies feel different. It is the same problem, one row up. *A dependency failure is a write by a foreign head to a shared region whose cut was never written down or written down wrongly, and its cost is the halt cost of the row at which the version was bound.* Pin, float, vendor, lockfile, monorepo and reproducible builds are choices of that row.

9.3 The Census

Decisions leave traces in pull requests, incident reports, design reviews and migration write-ups. Take a sample from a codebase's history, code each by operation, row and idealisation set, and record whether coding was possible. Decisions that code onto one idealisation are the simple ones. Those that code onto several are the composed ones. Those that do not code at all are either a ninth idealisation or a coding error, and either way they are the interesting residue. It is days of work for any team whose history is written down, and it is Prediction 1. A decision with a shape can be judged, and judging it means judging the rule it was made with.

10 The Test

The list of things a rule can be answerable to is now complete. A rule can answer to a wall, to a bill, to a cost under an idealisation at some row, to a measurement, or to nothing but the fact that others follow it. That list makes one question askable of any imperative.

Test 1 (What is this practice answerable to?)**.** For any practice stated as an imperative, name which of the following it answers to, and the specific instance:

(a) A cost. Under which of the eight idealisations, paid at which row, with what measured ratio and threshold; where physics sends a bill, which bill.

(b) A theorem. Which one, with its fine print, and whether it licenses the practice or merely constrains it.

(c) A measured regularity. Which study, on what population, when, and whether replicated.

(d) Uptake. Nothing outside the population that adopted it; its value is coordination and its truth condition is that others follow it.

A practice offered with no answer is an assertion that something is optimal with no statement

of what it is optimal for, and there is nothing in it to check.

A verdict of (d) is not a condemnation: driving on the left answers to nothing but the fact that others do, and it is among the most valuable rules there are; what (d) forbids is defending a convention as a discovery. Vincenti established that a whole category of engineering knowledge is prescription without theoretical grounding [100, 1]. What he could not say, having no physical layer to check against, is which prescriptions have a cost underneath them. Sections 5 to 7 are that physical layer.

10.1 Four Rules

Four practices, chosen because their answers differ.

Keep the working set in memory. Verdict (a): the latency bound, paid at the system row, and quantified because Gray and Putzolu wrote the conditions down. In 1987 Gray and Putzolu derived the break-even interval for keeping a page in memory as $A\$/(M\$ \cdot B)$, the price of a disk access per second over the price of a byte of memory times the page size. Their inputs gave 400 seconds, hence the five-minute rule [37]. In the same paper is the sentence this paper is about:

> "These results depend on current price ratios of processors, memory and disc accesses. These ratios are changing and hence the constants in the rules are changing." [37]

Because the formula was public, everybody could watch it expire. In 1997 it gave 266 seconds, still five minutes only because page size had grown eightfold and absorbed a price change [38]. In 2007 flash split the question into three answers [36]. In 2019 memory against disk reached four hours and flash against disk a day, mechanism stated [8]. A 2025 preprint that charges for processor time reports seconds [106]. This is a fully anchored practice, and there are very few.

Do not optimise prematurely. Verdict (a), with the regime lapsed. Since the subject is dropped conditions, here is the whole paragraph:

> "There is no doubt that the grail of efficiency leads to abuse. Programmers waste enormous amounts of time thinking about, or worrying about, the speed of noncritical parts of their programs, and these attempts at efficiency actually have a strong negative impact when debugging and maintenance are considered. We *should* forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil. Yet we should not pass up our opportunities in that critical 3%. A good programmer will not be lulled into complacency by such reasoning, he will be wise to look carefully at the critical code; but only after that code has been identified." [56]

Only the answer travelled. The cost it answered, debugging and maintenance time, is in the first two sentences and did not travel, which is why the proverb now supports positions Knuth argued against. The regime lapsed too. The procedure, identify the critical code by measuring and only then optimise it, assumes running cost is dominated by instruction count. Since then a cache miss has become 250 to 420 cycles [82, 18] and a memory fetch ten thousand times the energy of an add [22], so the expensive decision is now made when the data structure is chosen, before there is anything to profile [2]. The principle survives; the procedure does not.

Keep lines to eighty columns. Verdict (d). Hollerith's 1890 card was sized to fit boxes made for the Treasury Department [50]. IBM's 1928 rectangular holes packed eighty columns into that width [48]. Terminals inherited the number, and mid-1970s terminals shipped at many other widths, so no display forced it [84]. The numbers have since drifted, PEP 8 at 79, Google's Java guide at 100, Black at 88, Go at none, and the practice is still enforced. Reading research on prose finds that longer lines read faster and that preference disagrees with performance [27, 11, 69]. On code there is nothing; the nearest work trains on readability ratings, which is circular [16]. The rule is a convention, and a line limit is worth having; what the test exposes is that nobody has measured which limit, and that the one in use is the width

of a card.

Limit work in progress. Verdict (b), and the theorem does not license the practice. Little's law, $L = \lambda W$, where L is the work in progress, λ the throughput and W the time each item spends in the system, holds under almost no hypotheses [62, 92], so the relationship is not negotiable. But it is an identity, not a mechanism. Cut L, and the identity is equally consistent with W falling at constant λ, which the advocacy promises, and with λ falling at constant W, which nobody advertises. The practice's whole empirical content sits in a constant-throughput clause the theorem says nothing about, and that is why the practice literature's own assumption lists are stronger than the theorem's [7, 97]. The mechanism that would justify it, Kingman's heavy-traffic result [53], carries exactly the assumptions software development violates. The one empirical study, of 8,000 work items, called for replication and has not had it [89].

Practice	Verdict	What it is answerable to
Keep the working set in memory	(a)	The latency bound; a formula published in 1987 and recomputed four times since
Do not optimise prematurely	(a), lapsed	Instruction count against maintenance time; the denominator changed
Eighty-column limit	(d)	A card whose width a Treasury box fixed in 1890; nothing measured on code
Limit work in progress	(b), insufficient	Little's law, an identity that cannot license the intervention

10.2 Contradictory Proverbs

The field's proverbs come in pairs that seem to contradict, and do not. Each pair is one inequality seen from both sides: a case analysis whose separating condition was dropped from both branches.

Pair	The two costs	Which side a system is on
DRY against decouple	A change made twice, against a boundary coupling two sources of change	Do the two copies change together or separately in the history
YAGNI against design for change	Unused flexibility, against retrofitting it	How likely is the change, and what does reversal cost
Small services against modular monolith	Coordination cost of a shared deployable, against operational and latency cost of a boundary	Releases last quarter that needed two teams; fraction of tail latency in remote calls
Normalise against denormalise	Write and consistency cost, against read and join cost	Read-to-write ratio, and whether reads span the join
Fail fast against be liberal	A wrong result, against a refused request	Who bears the failure, and is the input under the team's control
Explicit against convention	Stating it, against a reader inferring it	How many readers, how expert, how often
Pin against float	Reversal cost of a foreign write, against staleness and security debt	How often the region read from changed across the cut

The third column is instructions, and most of them can be measured on a given system within a week. Put the costs back and the principles of software engineering come out, each one the cheaper side of an inequality between two costs, so a unit of engineering knowledge has a normal form: *for decisions of shape (operation, row, idealisation set), alternative A is cheaper than B when this inequality between these two measurable quantities holds, and the observable that tells which side a system is on is this.* A theorem is the case where the inequality holds in every regime. A lookup is the case where there is no alternative. A convention is the case where the only cost on either side is the cost of differing from everyone else. Two entries of the third column cannot be measured within a week: who bears a failure is a judgment about people, and the likelihood of a future change is a different kind of quantity, the subject of §13. Which leaves the question of why the conditions were ever dropped.

11 How Conditions Are Lost

Conditions are lost by four mechanisms. None is a failure of character, and they compound. The first is skill acquisition, which compiles declarative knowledge into procedures that fire on a pattern, so that the reason is no longer retrievable from the procedure [6]. The second is that expert decision is recognition, not comparison. Fireground commanders compared options in fewer than twelve percent of 156 decision points [55], and software designers consider fewer options the more structured the decision [104], so the cost model that once made the action cheapest never enters working memory. The third is that making context explicit is expensive, and the person who pays is not the person who benefits [83, 40]. The fourth is formalisation itself: turning an argument into an equation keeps the variables and drops the constants and the setting. Amdahl's 1967 paper has no equations and one empirical constant, data management overhead near forty percent of instructions [5]. The algebra everyone calls Amdahl's law kept the equation, dropped the constant and silently fixed the problem size, and Gustafson needed a page and a half, twenty-one years later, to put the condition back [42]. The latency bound of §6 has travelled the same way: the classical model justifies its cost function by circuit delay [3], the textbook by locality and manufacturing [47], and the cube-root claim circulates in forum posts that cite nothing. Together these predict practitioners holding beliefs not indexed to their own projects' conditions, and a survey of 564 Microsoft engineers found exactly that [25].

The general form is old and belongs to economics: QWERTY is a practice optimal for a lapsed regime, retained by convention and network effects [23, 9], and inside software the same thing is the vaporisation of architectural knowledge [49]. This paper adds only the expression of the cached rule as a minimisation over a stated cost model, so that the regime a rule assumed becomes a quantity and a threshold rather than a story.

If that is right, a practice should invert when its inequality flips, with the same kind of people giving the same kind of reason in the opposite direction. Migrations are written up in public, so this can be checked.

11.1 Inversions

Case	The ratio that moved	What the practitioners said
Store against recompute	Instruction cost against memory stall	Acton 2014, with the cache-line accounting [2]
Buy hardware against optimise	Free performance against engineering cost; Dennard ended 2007 to 2012	Python to hand-tuned vector code is 62,806× on one kernel [60]

Case	The ratio that moved	What the practitioners said
Normalise, denormalise, back	Developer time against machine throughput, twice in opposite directions	Codd's opening obligation [20]; Stonebraker's 900 against 160,000 messages per second [93]; F1 calling eventual consistency "an unacceptable burden to place on developers" [85]
Monolith, services, monolith	Coordination cost of a shared deployable against operational and latency cost of a boundary	Segment [72]; Istio [66]; Uber, fifty services across twelve teams for one root cause [35]
Threads against processes	Argued to have crossed; transition cost priced higher	Lane: "I think this will be a disaster... largely silently" [58]
Site isolation	Nothing moved; a premise was falsified	After Spectre "it is no longer safe to render documents from different web sites in the same process" [78]
Cloud and back	Rent against own, conditions stated by the practitioner	Heinemeier Hansson: $3.2m budget, $1.5m projected saving, and the regime under which renting stays right [26]

Three things the table cannot carry. Site isolation bounds the framework, because inversions of that kind come from outside any cost model. And two counter-cases. Prechelt and colleagues found a design pattern's flexibility probably worth having absent a clear reason otherwise, and could identify the exceptions only by "software engineering common sense" [76]; some cached answers are robust across a regime no system leaves. Menzies and colleagues found no evidence across 171 projects that defects get more expensive to fix the longer they linger [67], a belief that has justified a great deal of process. The table records inequalities that have already flipped; the ratios still moving say which flip next.

11.2 Hypotheses

Each hypothesis below names the ratio that moves, what should invert, what should not, and an observable.

Hypothesis 1 (Cheap memory splits the memory practices in two). Memory cost per byte has fallen about six orders of magnitude since 1987, so every practice anchored on the price of a byte inverts. But the bound is on distance and more memory is a larger x, so practices anchored on the distance to a byte, locality, sequential access, data-oriented layout, intensify. *Observable*: the ratio of slowest to fastest in-memory access on a machine across generations; flat or falling while capacity rises refutes this. Half the evidence is in: the 2019 recomputation pushed the byte-price crossover to four hours [8]; the distance half awaits the observable.

Hypothesis 2 (Returns concentrate in packaging and near-data computation). $\Omega(x^{1/d})$ cannot be bought off, but it has two letters that can be moved. Raise d by stacking, which changes the exponent from a half to a third; at a billion cells that is worth a factor near thirty, and there is no fourth dimension after it. Or lower x by moving computation to the data. *Observable*: the share of a hardware generation's improvement attributable to organisation and placement rather than device speed.

Hypothesis 3 (Distrust practices track fleet size rather than device quality). Per-device error rates fall every generation and the practices built on distrust have not relaxed in thirty years, because the governing quantity is rate times count and the count has risen at least as fast. They

relax first where fleets stop growing: single machines, edge and embedded work. *Observable*: rate times count for the system actually run, now and five years ago.

Hypothesis 4 (Practices re-anchor from time to energy). The Landauer floor limits nobody; heat removal limits everybody and stopped improving when Dennard scaling ended. Rules justified by latency will be restated with energy as the objective, and the restatement changes answers about idling, batching and speculation. *Observable*: joules per unit of work on the hot path, which most teams do not measure.

Hypothesis 5 (Gap practices expire and floor practices do not). Between the fibre floor and delivered pings there is a factor of about three to win. Practices that hide the gap, prefetching, connection reuse, edge caching, work against a finite shrinking margin; practices that respect the floor, partitioning by geography, asynchronous defaults, a deliberate consistency model, are permanent. *Observable*: a system's median round trip against the great-circle fibre floor for the same endpoints.

Hypothesis 6 (The binding constraint moves from production to reversal). The cost behind idealisation 6 has no floor, which is why it has fallen as far and as fast as it has. A large part of good practice answers to that cost and to nothing else: do not repeat yourself, abstraction to avoid retyping, style guides, small reviewable changes, "we cannot afford to rewrite it". The halt column does not fall with it. So as writing gets cheap, the practices that survive are the ones that minimise the row at which decisions bind, not the amount of code. DRY weakens. YAGNI strengthens. Feature flags and late binding rise in value against anything whose benefit was saved typing. *Observable*: the fraction of a team's change cost attributable to writing, against reversal and verification, across its last twenty changes. If writing collapses and practice does not reorganise around reversal, either those practices never answered to production cost or the mechanism is wrong.

The last hypothesis is the hinge, because the cost it names is the one now being removed.

12 The Cost of a Decision

What happens when a cost is removed can be computed, because a decision has a cost with four terms. For a chosen alternative a,

$$C(a) \;=\; P(a) \;+\; V(a) \;+\; p_{\text{wrong}}(a) \cdot H(\text{row}(a)) \;+\; I(a).$$

Here P is production, what it takes to write the change. V is verification, what it takes to establish that the change does what was intended. p_{wrong} is the chance the decision is wrong given what is known when it is made, and H is the halt cost of the row it binds to, so their product is the expected cost of undoing it. I is information, what is spent reducing p_{wrong} before committing. The interpreter picks the a that minimises C, and the field's history reads as a record of which term dominated. For most of that history P was large. Where p_{wrong} and H were both high, practices minimised their product: feature flags, canaries and staged rollouts bind a decision temporarily at a row with lower H while p_{wrong} is still being reduced. The formula also says when to stop and ask somebody: when the reduction in $p_{\text{wrong}} \cdot H$ that the answer would bring exceeds I, the cost of getting it; and I depends on who is asked, so it also says whom, the cheapest holder of the answer.

12.1 Production Goes to Zero

Set P to nearly zero, as a capable code generator does. Every practice that was minimising P stops being answerable to anything. Any decision whose idealisation set was $\{6\}$ alone loses its trade-off, and duplication, boilerplate and much of what style guides exist for fall in that

class. What survives pays under something other than 6. Idealisations 1 to 4 are physics at the machine rows and do not move. Number 5 gets worse, because more heads are writing. Numbers 7 and 8 are untouched, because a model that writes cheaply does not know what the founder meant or what the world will want next year. What is left is

$$C(a) \;\approx\; V(a) \;+\; p_{\text{wrong}}(a) \cdot H(\text{row}(a)) \;+\; I(a),$$

every term about being wrong rather than producing. This is the second tier of §3.1 becoming visible: the software intention was always what was being decided, and the code was the receipt.

12.2 Verifying Decisions

Current verification assumes P is large: a person writes slowly and a reviewer reads what was written. It checks the artefact. That made sense when the artefact was the expensive thing, and it stops making sense when the artefact is cheap and regenerable, because reading is now the expensive step and it verifies the wrong object. What needs verifying is the decision, and §9 says what one is, so there are four checks. The *idealisation set* was identified correctly, so that the trade-off priced is the one the world charges for; the dependency failure of §9.2 fails this check. The *inequality's inputs were measured* on this system rather than carried over from someone else's. The *row is the lowest* that satisfies the constraint, since binding higher buys reversal cost that was not needed. And the *reversal path has been exercised*, because a rollback that has never run is a hope.

The checks split along the idealisations. Measured inputs and a working reversal path are checked by machines against machines. Whether the idealisation set is right is mechanical for 1 to 4, which have cost models. For 5 and 6 it is one head checking another's sheet and another's premises, which a machine does without tiring at the thousandth call site. For 7 and 8, which concern what the world wants, it is not mechanical at all. Whether the row is right depends on who depends on the binding. So the physical idealisations are verified by machines against measurements, the boundary idealisation by machines against each other, and the world idealisations by whoever owns the goal, who need not be an engineer: finance, for whether invoices may be altered; compliance, for whether data may leave the region; the founder, for simplicity now against flexibility later.

Each moment a machine doing engineering work stops to ask a person can be sorted under an idealisation in the same way. Information that exists but is expensive to reach at the organisation row is 2. An objective nobody has decided is 7, the residue that can only be routed, never automated. An unknown dependency is 5. A missing abstraction is 6, an ordinary engineering problem. Over thousands of interventions the residue, the stops that remain, is then not a mystery called judgment but a distribution over eight categories with a known direction: 1 to 4 already machine-checkable, 5 and 6 shrinking with tooling, 7 and 8 where the engineer stays. That is Prediction 7.

12.3 The Record

Both the verification above and the test need a written record of a rule with its conditions attached, and no existing format holds one. The pattern form asks for context and forces, qualitatively [4, 32]. Architecture decision records ask, in one line, that changes of context be noted [74]. Neither asks for a ratio, a threshold or a date.

Field	What goes in it
Rule	The imperative, as people say it.
Shape	Operation, row, idealisation set.

Field	What goes in it
Answerable to	(a) cost, (b) theorem, (c) measured regularity, or (d) uptake, with the instance named.
The two costs	Named as measurable quantities, not virtues.
Inequality	The condition under which the rule is the cheaper option.
Crossover	The value at which it stops being cheaper, with the inputs used.
Observable	What is measured on *this* system to know which side it is on.
Reversal	What undoing costs at the bound row, and whether the path has been exercised.
Inversion	What the rule becomes on the other side. Inability to state it means a theorem, a regime wider than anything the holder will build, or an unexamined belief.
Last recomputed	A date, because a verdict can change when a price ratio crosses, and a rule with no date is a rule nobody has checked.

The record serves two readers. A person following a rule cannot usually say when it would be wrong. The record writes the regime beside the rule, with a date, so that following and checking become separate activities. A machine told to prefer small services can only pattern-match. Given the record, it can measure the named observable and say which side of the inequality the system is on, and that matters most where the advice is wrong. And the machine hands back the record, and a person verifies it when the code is no longer worth reading. That is as far as the record goes, and the limit is exact.

13 The Limit

A practice resting on a physical cost can be recorded and recomputed, because the cost has a model. The other three idealisations have no cost models, 7 and 8 have no theorems, and nobody has written down what has to be preserved there. That is a fact about the type of the quantity, not a failure of effort. Every episode in which generation became mechanical has had the same shape, and it is the shape §4 predicted: somebody made a piece of reality hold still by writing down what had to be preserved, and generation followed. The C standard's "(only) the observable behavior of the abstract machine" made optimisation a discipline [17]. Serial equivalence made concurrency control mechanical [29]. Relational algebra made query planning mechanical, and Codd opened by stating the preserved quantity as an obligation to users [20]. The five-minute rule is the same fact where reality does not hold still: the quantity was written down, the prices moved, and the answer was recomputed each time.

Architecture is where this never happened, because its preserved quantity is a bet about the future. Parnas said so in 1972: begin with a list of the design decisions *likely to change*, and hide each in a module [75]. The list is not derived from the code. It is a person's guess about what the world will ask for next. Two companies holding byte-identical code, one serving a process unchanged for a decade and the other a market that reinvents itself quarterly, need different decompositions of the same bytes, and nothing computed from the bytes says which decomposition is right.

Object	A formula over	Where the inputs live
A program	the tape	in the text; fully present
An optimisation	the machine model	in a published cost model
A practice	two costs and a ratio	in nobody's document, but derivable by measurement
An architecture	a distribution over future requests	in nobody's document, and not derivable by measurement

The last row is why the engineer stays at idealisations 7 and 8.

14 Rival Explanations

The account is now complete, and it is worth holding only if the rival accounts fail. Each of the four below is partly right, and this paper keeps the part that is. What none of them does is say where a rule stops being right.

Suppose first that the rules of the field are conventions all the way down: index the filtered column because others do, and for no other reason. Then a rule would change only when the people changed, and it would track no quantity. But the practices of §11.1 reversed in public, several of them in both directions, with the practitioners naming the number that had moved. And where a formula was published, the number was recomputed four times as the prices moved while the rule's name stayed the same. Conventions exist, and the test has a category for them. A convention, though, has no number to track, and these tracked one.

Suppose the opposite: that engineering is applied computer science, and every sound rule is a theorem in working clothes. Then a theorem would be found under every sound rule. The census finds theorems under idealisations 6 and 4 and none under 7 and 8, which between them generate requirements, operations, maintenance and migration. And of the four practices examined, the practice that does rest on a theorem is the practice the theorem cannot license. Walls are theorems, and walls do not choose among the options they leave open.

Suppose instead that engineers are not minimising cost at all, but doing what social organisms do: keeping their standing, following taste, protecting the story of who they are. The paper takes this explanation most seriously, because much of it is true. Category (d) of the test is that observation written down, and §11 is about rules held for social reasons after the cost reasons have gone. What it cannot explain is the direction of change. The migration write-ups record engineers overruling their own group's habit and giving a measured number as the reason. Standing decides which rules a person holds between crossings; it does not decide where the crossings are. And taste, examined, is a rule whose conditions the holder can no longer state, which is a fact about memory rather than about the absence of cost.

Suppose last that the machine does not matter, and software is social all the way down. Then nothing about the machine would show up in every system regardless of who built it. But every machine larger than its fastest memory has a hierarchy, and the reason it cannot be engineered away is a theorem about distance. Every system that spans a network must choose what it gives up during a partition, by a theorem about messages. And the three bills that bite, moving, failing and stepping, are paid by every team on every continent. The machine sets the floor and the people build on it, and an explanation that dropped either half would explain half the field.

15 What This Forbids

An account that forbids nothing is worth little. Each of the following is a sentence somebody currently says, and the account rules it out. "This is a best practice", with no answer to

Test 1, is an assertion of optimality with nothing named to be optimal for. “This practice is universal” means one of the three things the record’s inversion field lists, and the holder should say which. “We tried it and it worked” is no evidence about the boundary, because under a favourable regime nearly anything works. “Physics requires it” is ruled out above the four bills; multicore is the standing example. “This is a judgment call” as a terminal explanation is ruled out, because every judgment call has an operation, a row and an idealisation set, and naming them is where the explanation begins. Verifying machine-written code by reading it is ruled out as a default, because it verifies the code, whose cost went to zero. Treating dependency management as a lookup is ruled out, because the failures come from 5 and 8 and the practice addresses 6.

16 Threats to Validity

The completeness of the eight, and so of the grammar, is a failed search, the position Turing occupied with respect to his own thesis. The largest weakness is that the census of §9.3 has not been run: the paper’s central empirical claim could have been tested before publication, in days, and was not. The residue coding of §12.2 is a coding of a catalogue, not a measurement. The latency bound is verified from the two published abstracts, which state the physical assumptions and call the hierarchy’s latency optimal, not from the full proof. The inversions table is a curated sample, not a population, and two counter-cases are recorded. The four verdicts are four; a fifth could go the other way. No single decision is followed through the grammar, the test, the formula and the record on one real system with real numbers, which is the largest gap in explanation as the unrun census is the largest gap in evidence. The term software intention overlaps the reference model’s specification and could be argued to be nothing more; if so, the claim that survives is that the software intention has the coordinates given here.

17 Predictions

The hypotheses of §11.2 carry their own observables; these are the predictions the framework makes about itself.

Prediction 1 (Every decision codes onto the eight). A sample of decisions from a project’s history, coded by operation, row and idealisation set, will code completely, and the hardest decisions will have the largest sets. A substantial uncodable residue refutes the grammar.

Prediction 2 (Every two-way practice has an inequality). For any practice with documented two-way traffic, two nameable, measurable costs govern it, and the inversion follows the crossover computed from independent cost series by a lag of years; Sutter’s self-reported lag of one to two years is one data point [94]. Failure to find the two costs on a majority of the cases of §11.1 refutes the central claim.

Prediction 3 (Genre predicts whether conditions survive). Practices published with a formula get recomputed; practices published as imperatives get argued about; the correlation is with the form of the original document, not the subject.

Prediction 4 (The condition gap is large and socially sourced). Practitioners asked to state a practice they follow will do so immediately; asked when it would be wrong they will do markedly worse, and the gap will be larger for practices learned socially than for practices derived themselves.

Prediction 5 (Halt cost dominates change cost). The cost of a change is dominated not by the size of the diff but by the halt cost of the highest row of §8 at which it binds a decision.

Prediction 6 (Records beat imperatives, for machines). An agent given practice records with ratios and observables will outperform an agent given the same practices as imperatives, with

the margin concentrated where the regime does not hold. A margin where the regime does hold means the mechanism proposed here is wrong.

Prediction 7 (The residue moves in one direction). When a machine doing engineering work is deployed and every human intervention is coded by idealisation, the share at 1 to 6 falls as tooling improves and the share at 7 and 8 does not. If it also falls, preferences and forecasts are recoverable from artefacts, and §13 is wrong.

Take any principle software engineering teaches and try to name what it is answerable to under Test 1 and which idealisations its alternatives pay under. Most will code. Some may not: a principle for which, after looking, no wall, no bill, no idealisation, no measurement and no convention can be found. Those are the only things in software engineering this paper cannot explain. Either they reveal a ninth cost, or they reveal the interpreter doing something other than minimising cost.

18 Conclusion

Turing described a computer exactly by setting every cost of computing to zero, and software engineering grew in the space he cleared. The engineer is an interpreter between people who want things and a machine that does what its sheet says, and the software intention the interpreter produces is shaped by a few walls and by the prices of everything else, the eight costs the model idealised away. Nothing is chosen in a machine where nothing costs anything, so every real decision pays differently under those eight, and the principles of software engineering are the cheaper sides of inequalities whose conditions can be recovered from the inventory. A decision costs writing, checking, the chance of being wrong times the cost of undoing, and finding out. The cost falling fastest is the first, the one physics never priced; when it goes, what is left is about being wrong, the engineer's place is where the missing input is a preference or a forecast, and what gets verified changes from the code to the decision. The claims are held as Turing held his thesis, by a search for exceptions, and the exceptions are the invitation.

Supplementary Materials

A companion document, *How the Sources Were Read*, records for every reference below where in this paper it is used, what it was taken to support, how it was interpreted, and where that interpretation could be mistaken; where the paper leaned on a formal result without explaining it, the explanation is there. It is published with the paper as a PDF and as a page where each entry can be opened, the source followed, and an objection filed against that entry alone; the same page lets a reader bring a situation and have the paper's concepts applied to it, and file what did not fit. Objections and exceptions that survive change the paper, with credit. Paper and companion live at `https://github.com/Onyangoe21/conceptual-space-of-software-architecture` and are rendered at `https://curiousedwin.com/publications/what-computation-costs`.

References

References

[1] Abran, A., Meridji, K., Dolado, J.J. (2006). Software engineering from an engineering perspective: SWEBOK as a study object. *UPGRADE/Novática* 7(1):46–52.

[2] Acton, M. (2014). Data-oriented design and C++. CppCon 2014. Slide text from the official deck.

[3] Aggarwal, A., Alpern, B., Chandra, A.K., Snir, M. (1987). A model for hierarchical memory. *Proc. STOC 1987*.

[4] Alexander, C., Ishikawa, S., Silverstein, M. (1977). *A Pattern Language*. Oxford University Press.

[5] Amdahl, G.M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conf. Proc.* 30:483–485.

[6] Anderson, J.R. (1982). Acquisition of cognitive skill. *Psychological Review* 89(4):369–406.

[7] Anderson, D.J., Carmichael, A. (2016). *Essential Kanban Condensed*. Lean Kanban University Press.

[8] Appuswamy, R., Borovica-Gajic, R., Graefe, G., Ailamaki, A. (2019). The five-minute rule 30 years later. *Communications of the ACM* 62(11).

[9] Arthur, W.B. (1989). Competing technologies, increasing returns, and lock-in by historical events. *Economic Journal* 99:116–131.

[10] Bennett, C.H. (1973). Logical reversibility of computation. *IBM Journal of Research and Development* 17(6):525–532.

[11] Bernard, M., Fernandez, M., Hull, S. (2002). The effects of line length on children and adults' online reading performance. *Usability News* 4(2).

[12] Bérut, A., et al. (2012). Experimental verification of Landauer's principle linking information and thermodynamics. *Nature* 483:187–189.

[13] Bilardi, G., Ekanadham, K., Pattnaik, P. (2009). On approximating the ideal random access machine by physical machines. *Journal of the ACM* 56(5), Article 27. Earlier: *Proc. SPAA 2002*, pp. 109–116, which states the $\Omega(x^{1/d})$ lower bound.

[14] Bozkurt, I.N., et al. (2017). Why is the Internet so slow?! *Proc. PAM 2017*.

[15] Brooks, F.P. (1987). No silver bullet: essence and accidents of software engineering. *IEEE Computer* 20(4):10–19.

[16] Buse, R.P.L., Weimer, W. (2010). Learning a metric for code readability. *IEEE TSE* 36(4):546–558.

[17] ISO/IEC 9899:2011, draft N1570. Observable behaviour §5.1.2.3 ¶6.

[18] Lam, C. (2022). AMD's Zen 4, part 2: memory subsystem. Chips and Cheese, 8 November 2022.

[19] Church, A. (1936). An unsolvable problem of elementary number theory. *American Journal of Mathematics* 58(2):345–363.

[20] Codd, E.F. (1970). A relational model of data for large shared data banks. *Communications of the ACM* 13(6):377–387.

[21] Conway, M.E. (1968). How do committees invent? *Datamation* 14(4):28–31.

[22] Dally, W. (2022). On the model of computation: point. *Communications of the ACM* 65(9):30–32.

[23] David, P.A. (1985). Clio and the economics of QWERTY. *American Economic Review* 75(2):332–337.

[24] Dennard, R.H., et al. (1974). Design of ion-implanted MOSFETs with very small physical dimensions. *IEEE JSSC* 9(5):256–268.

[25] Devanbu, P., Zimmermann, T., Bird, C. (2016). Belief & evidence in empirical software engineering. *Proc. ICSE 2016*.

[26] Heinemeier Hansson, D. (2023). We have left the cloud. world.hey.com, 23 June 2023.

[27] Dyson, M.C. (2004). How physical text layout affects reading from screen. *Behaviour & Information Technology* 23(6):377–393.

[28] Earman, J., Norton, J.D. (1998–99). Exorcist XIV: the wrath of Maxwell's demon. *Studies in History and Philosophy of Modern Physics*, parts I and II.

[29] Eswaran, K.P., Gray, J.N., Lorie, R.A., Traiger, I.L. (1976). The notions of consistency and predicate locks in a database system. *Communications of the ACM* 19(11):624–633.

[30] Deutsch, L.P., and colleagues (c. 1994). The fallacies of distributed computing. Explicated in Rotem-Gal-Oz, A. (2006).

[31] Fischer, M.J., Lynch, N.A., Paterson, M.S. (1985). Impossibility of distributed consensus with one faulty process. *Journal of the ACM* 32(2):374–382.

[32] Gamma, E., Helm, R., Johnson, R., Vlissides, J. (1994). *Design Patterns*. Addison-Wesley.

[33] Gandy, R. (1980). Church's thesis and principles for mechanisms. In *The Kleene Symposium*. North-Holland.

[34] Gilbert, S., Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News* 33(2):51–59.

[35] Gluck, A. (2020). Introducing domain-oriented microservice architecture. Uber Engineering Blog, 23 July 2020.

[36] Graefe, G. (2007). The five-minute rule twenty years later. *Proc. DaMoN 2007*; republished *ACM Queue* 6(4), 2008.

[37] Gray, J., Putzolu, G.R. (1987). The 5 minute rule for trading memory for disc accesses. *Proc. ACM SIGMOD 1987*, pp. 395–398.

[38] Gray, J., Graefe, G. (1997). The five-minute rule ten years later. *ACM SIGMOD Record* 26(4):63–68.

[39] Grier, D.A. (2005). *When Computers Were Human*. Princeton University Press.

[40] Grudin, J. (1996). Evaluating opportunities for design capture. In *Design Rationale*. Lawrence Erlbaum.

[41] Gunter, C.A., Gunter, E.L., Jackson, M., Zave, P. (2000). A reference model for requirements and specifications. *IEEE Software* 17(3):37–43.

[42] Gustafson, J.L. (1988). Reevaluating Amdahl's law. *Communications of the ACM* 31(5):532–533.

[43] Hayden, P., Wang, J. (2025). On the Bekenstein bound. *Quantum* 9:1664.

[44] Hennessy, J.L., Patterson, D.A. (2019). A new golden age for computer architecture. *Communications of the ACM* 62(2):48–60.

[45] Hodges, A. (1983). *Alan Turing: The Enigma*. Burnett Books.

[46] Hong, J., Lambson, B., Dhuey, S., Bokor, J. (2016). Experimental test of Landauer's principle in single-bit operations on nanomagnetic memory bits. *Science Advances* 2(3):e1501492.

[47] Hennessy, J.L., Patterson, D.A. *Computer Architecture: A Quantitative Approach*, ch. 2.

[48] IBM. The punched card. ibm.com/history/punched-card.

[49] Jansen, A., Bosch, J. (2005). Software architecture as a set of architectural design decisions. *Proc. WICSA 2005*.

[50] Jones, D.W. Punched cards: a brief illustrated technical history. University of Iowa.

[51] Jun, Y., Gavrilov, M., Bechhoefer, J. (2014). High-precision test of Landauer's principle in a feedback trap. *Physical Review Letters* 113:190601.

[52] Kalia, A., Kaminsky, M., Andersen, D.G. (2019). Datacenter RPCs can be general and fast. *Proc. NSDI 2019*.

[53] Kingman, J.F.C. (1961). The single server queue in heavy traffic. *Math. Proc. Cambridge Phil. Soc.* 57(4):902–904.

[54] Kleene, S.C. (1936). Lambda-definability and recursiveness. *Duke Mathematical Journal* 2:340–353.

[55] Klein, G.A., Calderwood, R., Clinton-Cirocco, A. (1986). Rapid decision making on the fire ground. *Proc. Human Factors Society* 30(16):576–580.

[56] Knuth, D.E. (1974). Structured programming with go to statements. *ACM Computing Surveys* 6(4):261–301. The passage is at p. 268.

[57] Landauer, R. (1961). Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development* 5(3):183–191.

[58] Lane, T. (2023). Re: let's make PostgreSQL multi-threaded. pgsql-hackers, 5 June 2023.

[59] Lehman, M.M. (1980). Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE* 68(9):1060–1076.

[60] Leiserson, C.E., et al. (2020). There's plenty of room at the Top. *Science* 368(6495):eaam9744.

[61] Li, C., Ding, C., Shen, K. (2007). Quantifying the cost of context switch. *Proc. ExpCS 2007*.

[62] Little, J.D.C. (1961). A proof for the queuing formula: $L = \lambda W$. *Operations Research* 9(3):383–387.

[63] Lloyd, S. (2000). Ultimate physical limits to computation. *Nature* 406:1047–1054.

[64] Margolus, N., Levitin, L.B. (1998). The maximum speed of dynamical evolution. *Physica D* 120:188–195.

[65] McMullin, E. (1985). Galilean idealization. *Studies in History and Philosophy of Science* 16(3):247–273.

[66] Mendonça, N.C., Box, C., Manolache, C., Ryan, L. (2021). The monolith strikes back. *IEEE Software* 38:17–22.

[67] Menzies, T., Nichols, W., Shull, F., Layman, L. (2017). Are delayed issues harder to resolve? *Empirical Software Engineering* 22(4):1903–1935.

[68] Meza, J., Wu, Q., Kumar, S., Mutlu, O. (2015). A large-scale study of flash memory failures in the field. *Proc. SIGMETRICS 2015*.

[69] Nanavati, A.A., Bias, R.G. (2005). Optimal line length in reading: a literature review. *Visible Language* 39(2):121–145.

[70] Eskildsen, S. *napkin-math*. Single-host rows re-measured on GCP `c4-standard-48-lssd`, 8 March 2026.

[71] Naur, P. (1985). Programming as theory building. *Microprocessing and Microprogramming* 15(5):253–261.

[72] Noonan, A. (2018). Goodbye microservices. Segment Engineering Blog, 10 July 2018.

[73] Norton, J.D. (2005). Eaters of the lotus: Landauer's principle and the return of Maxwell's demon. *Studies in History and Philosophy of Modern Physics* 36:375–411.

[74] Nygard, M. (2011). Documenting architecture decisions. cognitect.com.

[75] Parnas, D.L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM* 15(12):1053–1058.

[76] Prechelt, L., Unger, B., Tichy, W.F., Brössler, P., Votta, L.G. (2001). A controlled experiment in maintenance comparing design patterns to simpler solutions. *IEEE TSE* 27(12):1134–1144.

[77] Proesmans, K., Ehrich, J., Bechhoefer, J. (2020). Finite-time Landauer principle. *Physical Review Letters* 125:100602.

[78] Reis, C., Moshchuk, A., Oskov, N. (2019). Site isolation. *Proc. USENIX Security 2019*, pp. 1661–1678.

[79] Rice, H.G. (1953). Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society* 74(2):358–366.

[80] Saltzer, J.H., Reed, D.P., Clark, D.D. (1984). End-to-end arguments in system design. *ACM TOCS* 2(4):277–288.

[81] Schroeder, B., Pinheiro, E., Weber, W.-D. (2009). DRAM errors in the wild: a large-scale field study. *Proc. SIGMETRICS 2009*.

[82] 7-cpu.com, Intel Skylake page. i7-6700 at 4.0 GHz, DDR4-2400 CL15.

[83] Shipman, F.M., Marshall, C.C. (1999). Formality considered harmful. *Computer Supported Cooperative Work* 8:333–352.

[84] Shirriff, K. (2019). IBM, sonic delay lines, and the history of the 80×24 display. righto.com, November 2019.

[85] Shute, J., et al. (2013). F1: a distributed SQL database that scales. *PVLDB* 6(11):1068–1079.

[86] Sieg, W. (1997). Step by recursive step: Church's analysis of effective calculability. *Bulletin of Symbolic Logic* 3(2):154–180.

[87] Simon, H.A. (1962). The architecture of complexity. *Proc. American Philosophical Society* 106(6):467–482.

[88] Simon, H.A. (1996). *The Sciences of the Artificial*, 3rd ed. MIT Press.

[89] Sjøberg, D.I.K. (2018). An empirical study of WIP in kanban teams. *Proc. ESEM 2018*.

[90] Soare, R.I. (1996). Computability and recursion. *Bulletin of Symbolic Logic* 2(3):284–321.

[91] Soares, L., Stumm, M. (2010). FlexSC: flexible system call scheduling with exception-less system calls. *Proc. OSDI 2010*.

[92] Stidham, S., Jr. (1974). A last word on $L = \lambda W$. *Operations Research* 22(2):417–421.

[93] Stonebraker, M., Çetintemel, U. (2005). "One size fits all": an idea whose time has come and gone. *Proc. ICDE 2005*.

[94] Sutter, H. (2005). The free lunch is over. *Dr. Dobb's Journal* 30(3).

[95] Turing, A.M. (1936–37). On computable numbers, with an application to the Entscheidungsproblem. *Proc. LMS* s2-42:230–265.

[96] Turing, A.M. (1950). Computing machinery and intelligence. *Mind* 59(236):433–460.

[97] Vacanti, D. (2015). *Actionable Agile Metrics for Predictability*. ActionableAgile Press.

[98] Valiant, L.G. (1990). A bridging model for parallel computation. *Communications of the ACM* 33(8):103–111.

[99] van Leeuwen, J., Wiedermann, J. (2001). The Turing machine paradigm in contemporary computing. In *Mathematics Unlimited: 2001 and Beyond*. Springer.

[100] Vincenti, W.G. (1990). *What Engineers Know and How They Know It*. Johns Hopkins University Press.

[101] von Neumann, J. (1956). Probabilistic logics and the synthesis of reliable organisms from unreliable components. In *Automata Studies*. Princeton University Press.

[102] Wulf, W.A., McKee, S.A. (1995). Hitting the memory wall: implications of the obvious. *ACM SIGARCH CAN* 23(1):20–24.

[103] Yan, L.L., et al. (2018). Single-atom demonstration of the quantum Landauer principle. *Physical Review Letters* 120:210601.

[104] Zannier, C., Chiasson, M., Maurer, F. (2007). A model of design decision making based on empirical results of interviews with software designers. *Information and Software Technology* 49(6):637–653.

[105] Zave, P., Jackson, M. (1997). Four dark corners of requirements engineering. *ACM Transactions on Software Engineering and Methodology* 6(1):1–30.

[106] Zhang, T., et al. (2025). Five-minute rule 40 years later. arXiv:2511.03944.